Muhammad Sohail

AI Full-Stack Engineer | ML Expertise | LLMs, AI Agents, RAG, MCP
TypeScript, React, Node.js, Python | AWS Certified

https://msohail.work
Firenze, Italy
✉ msohail.dev@gmail.com +39 392 803 6934
msohailse sohail0992 sohail

SUMMARY

AI full-stack engineer with 7+ years of experience building and running cloud-native applications with TypeScript/Node.js, React, Angular, Nuxt, PostgreSQL, AWS and Kubernetes. Now focused on production AI systems: LLMs and AI agents with tool use, RAG pipelines on vector databases, MCP servers and multi-agent engineering workflows, with cost caps, isolation, supervision and audit logging built in, plus ML classification models trained from scratch. Delivers live products from data model and API to UI, CI/CD and cloud infrastructure, including Flutter mobile apps on the App Store (iRide, HCAI), with an interest in impact-driven and sustainability-focused products.

EDUCATION

UiT The Arctic University of Norway Sep 2026 – Jan 2027
Master in Computer Science

- **Coursework:** Machine Learning (supervised and unsupervised learning, decision trees, neural networks) and Advanced Computer Security (information-flow control, side-channel attacks, fuzz testing and verification)

Università degli Studi di Firenze Sep 2024 – Sep 2026
Master's Degree, Software: Science and Technology

- **Coursework:** Architecture, Model and Analysis of Cyber-Physical Systems, Data Collection and Machine Learning for Critical Cyber-Physical Systems, Distributed Programming for Web, IoT and Mobile Systems, Natural Language Processing, Statistical Methods for Computer Science

Riphah International University 2014 – 2018
Bachelor of Science, Computer Software Engineering (CGPA 3.05)

Certifications

AWS Certified Cloud Practitioner	AWS
MongoDB Certified	MongoDB University
React Certification	TestDome
Dart and Flutter: The Complete Developer's Guide	Udemy
English Proficiency Test	Duolingo

WORK EXPERIENCE

Trident Marketing Sep 2018 – Present
Tech Lead / Software Engineering Manager

- Led a cross-functional engineering team delivering Angular and Node.js applications, coordinating backend, frontend, and QA development across multiple production systems.
- Architected scalable microservices and hybrid mobile solutions, leveraging Node.js, TypeScript, and MongoDB to support enterprise-level requirements. Designed system architecture pat-

terns, API gateways, and authentication flows (JWT/OAuth) for secure, cloud-native applications.

- Integrated mobile apps with external booking systems and third-party services to streamline business operations (REST, WebSockets, ElasticSearch).
- Designed and co-architected the TRA API to manage real-time data flow between multiple apps and external booking systems, leveraging REST API principles for scalability and efficiency (Node.js, TypeScript).
- Designed the TRADB (MongoDB) schema for large-scale data handling and integrations.
- Optimized system performance and scalability by managing real-time synchronization across multiple Elastic indices, applying caching strategies, and tuning database queries.
- Owned data operations for TRADB: built and ran 30+ repeatable Python jobs (pymongo, pandas, requests) for ETL from the booking system, large-scale backfills and data corrections, Elastic index rebuilds, payment-to-booking reconciliation, nightly inventory sync, and seeding dev environments from live records.
- Enabled booking-to-sales conversion by updating household records, converting leads to members, and indexing data in the Manifest System.
- Built the Engage App using Angular and TypeScript, enabling agents to track customer data (e.g., demographics, call history) and manage bookings within the same platform, significantly streamlining campaign workflows and improving agent productivity.
- Engineered the Manifest System (Angular, Node.js) to track bookings, agent performance, commissions, and the overall sales lifecycle.
- Data Visualization: Highcharts, Chart.js, Recharts, dashboard analytics, interactive reporting, real-time data visualization.
- Developed the DNQ System, a multi-part survey platform:
  - Backend uses a tree data structure for complex survey logic.
  - Frontend built with Ionic (Angular, Node.js).
- Contributed to the Lead Input Application (LIA) UI for uploading leads to S3 and splitting large PDFs (Angular, Node.js).
- Implemented a message broker for high-volume calls, integrating with TRADB for faster processing.
- Built WebSocket-based solutions for real-time notifications, integrated with REST APIs to enhance scalability (e.g., member reservations).
- *Quality Assurance*:
  - Engineered complex Data Merging Logic for Household/Lead entities, utilizing Mocha & Chai to validate 20+ edge cases (e.g., duplicate resolution, membership inheritance).
  - Implemented a robust Test-Driven Development (TDD) workflow using JSON fixtures to simulate API responses, ensuring zero-regression deployments, plus behavior-driven (BDD) tests in Python with Behave.
  - Developed the logic for Dynamic Meeting Payloads, validating attendee filtering rules (primary vs. secondary contacts) through automated assertion suites.
  - *CI/CD & Reporting*: Integrated the test suite into Jenkins pipelines, configuring it to capture JSON test results and generate automated build reports for immediate feedback on code stability.
- *Infrastructure & CI/CD*:
  - Optimized MongoDB cluster configurations, saving approximately $12,000 annually while maintaining system performance and reliability.

- Led Kubernetes-based deployments and CI/CD pipeline improvements (Jenkins, GitHub Actions), incorporating Infrastructure as Code (IaC) practices to enhance scalability, resilience, and operational efficiency.
- Run 30+ Node.js and Python microservices in production on Azure Kubernetes Service (AKS), with secrets managed in Azure Key Vault, Elastic APM monitoring across services, and Jaeger distributed tracing on high-volume consumers.
- Created unit/behavior tests (Selenium, Chai, Mocha, Cypress) and integrated GitHub pipelines (JavaScript, Node.js).

Iterca Platform Mar 2026 – Sep 2026
Senior Software Engineer / Platform Engineer

- **Cloud Infrastructure & CI/CD Automation:** Engineered a scalable deployment pipeline on AWS Amplify for a distributed Next.js architecture. Resolved complex serverless injection issues, ensuring secure delivery of runtime configurations to Node.js Lambdas.
- **Complex Business Logic & Ledgers:** Designed a double-entry financial and meeting credit ledger. Implemented strict database-level concurrency controls (locks) to successfully prevent race conditions and credit exhaustion exploits during checkout.
- **Modern Web Architecture (SSR/RSC):** Architected the platform using TypeScript, React, and Next.js App Router. Bridged React Server Components (SSR) with Client Components to optimize web performance, reduce payloads, and deliver responsive UI flows.
- **Database Management & Caching:** Provisioned AWS RDS PostgreSQL within a secure VPC. Balanced database load by implementing advanced Next.js Edge caching with custom invalidation rules.
- **Platform UX & Reusable Tooling:** Elevated usability by refactoring legacy forms into state-driven React components with comprehensive error boundaries to improve developer experience and user conversion flows.
- **AI & Workflow Automation:** Engineered a standalone MCP (Model Context Protocol) Server for secure AI-driven note management, and built an autonomous webhook state machine to automate meeting tracking and financial escrow payouts for 50+ coaches.

iRide PK Jan 2025 – Sep 2025
Flutter Developer (Contract/Part-Time)

- Built the iRide ride-hailing app in Flutter, reaching 60,000+ users in Lahore, Pakistan within 2 months of launch.
- Used BLoC and Dart null safety to deliver robust, testable state management.
- Reduced MongoDB hosting costs from $1,500/mo to $300/mo (80% reduction) by implementing Redis caching, optimizing indices, and tuning queries.

Code For Pakistan (Volunteer) Mar 2021 – Present
Web Engineer/Flutter Developer

- Integrated a WhatsApp chatbot for the COVID-19 Telehealth Initiative, enabling identification and random assignment of high-risk patients to volunteer doctors (React, Node.js, PostgreSQL).
- Built a Flutter-based mobile app for lab evaluations (LAARC-standard surveys) paired with a Vue.js dashboard for performance reporting (Flutter, Vue.js, Node.js).
- Collaborated with WHO and NIH as a Solutions Engineer to architect and implement secure healthcare applications (HCAI app) using Flutter, Vue.js, Node.js, and MongoDB. Applied system design principles, microservices architecture, and performance tuning while documenting architecture and workflows to improve healthcare protocols and post-surgery infection tracking.

- Education Mentor, KP Women Program / WomenInTechPK *(2023 – Present)*: conducted training sessions (e.g., Git and version control) for participants to enhance their collaborative development workflows.
- Facilitated the interview process in 2025 to hire mentors and candidates for the KP Women Program.
- Served as speaker and mentor for the KP Government Innovation Fellowship.

Jeddah Soft Sep 2018 – Dec 2018
Mern Stack Developer (Remote Part Time)

- *Doctor App* (MySQL, React, Node.js):
  - Built a React + Bootstrap dashboard for managing doctor appointments.
  - Implemented Highcharts for admin reporting.
  - Maintained a session-based shopping cart tied to user authentication.
- *ShopAPI (Open Source):*
  - See the ShopAPI GitHub repo (Express/Node.js + WebSockets).
  - Uploaded and indexed files/images via Cloudinary.
  - Maintained session-based shopping cart logic.

xFlow Research Inc. Oct 2016 – Feb 2017
Internee

- Developed a VR basketball game for HTC Vive.

AI AGENTS & AGENTIC AUTOMATION

Multi-Agent Engineering Team: Hermes Agents with Obsidian Memory
Hermes agents, Obsidian, GitHub, SonarCloud, Buzz (Nostr-style relay)

- Run a personal multi-agent pipeline in a shared channel, each agent keeping memory in an Obsidian vault: a CTO agent triages work and delegates to a developer agent (test-first, opens PRs), a PR-review agent (green build, tests and Sonar plus deep review, merges only after QA sign-off) and a QA agent (runs the app, read-only database access, production check after each merge). ApplyBot's failure watcher posts alerts to the channel and tags the CTO agent, which triages each one and hands it to a developer agent.
- Added a reverse-engineering agent that analyzes shipped artifacts statically and at runtime and returns cited evidence, an architecture map and a developer TODO list.

ApplyBot: Pay-per-Search AI Job-Matching Product
Next.js 15, TypeScript, PostgreSQL (RDS), Drizzle, AWS (Fargate, Lambda, Amplify), Amazon Bedrock, Better Auth, Vitest, GitHub Actions

- Designed, built and operate a production pay-per-search job-matching product at applybot.eu: a customer uploads a CV and sets target roles and places, a search finds jobs, an LLM on Bedrock scores each against the CV (per-search budget and token cap), and only strong matches are returned; the customer applies themselves.
- Isolated every paid search in its own AWS Fargate container started through a launcher Lambda, with only a run-scoped token and no database access, per-customer and global concurrency limits, and an admin control to stop one or all running searches.
- Implemented a central AI cost gate on every AI call, a scoped in-app support chat assistant (no tools, no database access, cost-capped per customer and IP) and a single-source money-back rule shared by UI, terms and refund API; tested with Vitest and Testcontainers, GitHub Actions CI, auto-deploy and a 25-check post-release script against the live site.

Multi-App Billing & Payments Platform with MCP Admin Server
TypeScript, AWS Lambda, PostgreSQL, Drizzle, Stripe, Wise, MCP, Cloudflare Workers, SonarCloud

- Built and run in production a reusable billing module (private npm package) and hosted service that is the single payment system behind live apps such as ApplyBot, called over HTTP (hosted checkout, invoices, subscriptions, refunds, token wallet). Multi-merchant by design, with a separate API key, Stripe account and webhook secret per app; an app without Stripe keys fails closed.
- Built an invoice and subscription ledger with an append-only events table, where settling an invoice, extending access and writing the ledger entry happen in one database transaction. Settlement is automatic for card payments (signed Stripe webhooks) and bank transfers (Wise statement matching that never guesses: a payment must name the invoice and cover at least 95%). Added discount codes (100% off, batches, product scoping), a TOTP-protected admin dashboard with audit log and a support module with secure attachments; about 420 tests run against a throwaway Docker Postgres.
- Built an MCP server as the owner's chat-side admin connector: payment and transaction lookup, discount-code management and a support inbox, with revocable, expiring, hashed personal keys, per-key rate limits and audit logging. It has no refund or tax tools, and customer text is treated as untrusted.

Agentic AI Scheduling ChatBot (Multi-Tenant SaaS)
TypeScript, Nuxt 4, LangChain.js, n8n, Amazon Bedrock, PostgreSQL + pgvector, RAG, Python, AWS (SQS, Lambda, EventBridge, EC2), Docker

- Built a multi-tenant SaaS where each business connects its own channels (WhatsApp number, email and website chat) and an LLM agent (LangChain.js, DeepSeek on Bedrock, RAG on pgvector) books, reschedules and cancels appointments, runs a waitlist and sends birthday offers, on an event-driven pipeline of a per-business WhatsApp bridge, a Nuxt API, SQS with dead-letter queues and a Lambda worker.
- Added n8n workflows with PostgreSQL session state and a RAG pipeline over product manuals and pricing data, so the agent keeps context across a customer's journey and gives grounded, accurate answers on every channel; also automated residential battery sizing from HomeWizard energy telemetry.
- Created an "Observer Harness", a separate Python service woken by Postgres triggers on every customer message and agent error, that escalates in tiers (check, steer with a hidden note, take over with a fallback reply, then draft a fix PR for human review). It fails open and runs with a minimal-privilege database role.
- Added agent guardrails (tool-call limits, per-customer and per-business reply caps, per-plan usage limits) and a Nuxt 4 owner dashboard; 200+ tests with scripted fake LLMs keep CI free of cloud credentials and LLM spend.

Document Intelligence & Automated Form Processing
Google Cloud Vision, OpenAI, AWS Pub/Sub, OCR

- Engineered an OCR pipeline (Google Cloud Vision, AWS Pub/Sub events) digitizing over 50k handwritten pamphlets at up to 50,000 images per day, with OpenAI entity extraction and KPI derivation pre-filling digital records.
- Created a dual-view interface showing the pre-filled form beside the pamphlet image so one operator confirms and submits.
- Developed a FastAPI rule engine that validates the OCR-extracted data against historical records to determine user eligibility and system compatibility.

Voice Intelligence & Real-Time Transcription
IBM Watson Speech-to-Text, WebSockets, Microservices

- Developed a WebSocket microservice streaming call recordings to Watson Speech-to-Text for real-time transcription and compliance monitoring, with a security layer detecting sensitive information in live audio.

Skills

- **AI, ML & Automation**: Machine Learning (PyTorch, scikit-learn, Classification, Supervised & Unsupervised Learning, Neural Networks, Model Training & Evaluation), LLMs, AI Agents, Agentic AI Development, Multi-Agent Systems, Tool Use, RAG, Vector Databases (pgvector), MCP, NLP, LangChain.js, Amazon Bedrock, OpenAI, DeepSeek, n8n, Prompt-Injection Hardening, LLM Cost & Safety Guardrails, Google Cloud Vision, IBM Watson (STT)
- **Languages**: TypeScript, JavaScript, Python, Java, Go, Dart, C++, SQL
- **Frameworks**: React, Next.js, Nuxt, Angular, Vue, Ionic, Node.js, Express, FastAPI, Flutter, Quarkus, Drizzle ORM, Pandas, NumPy, Vitest, Testcontainers, Behave, Mocha, Chai, Cypress, Selenium, Protractor, Highcharts, Recharts, Bootstrap
- **Databases**: PostgreSQL, MongoDB, MySQL, MsSQL, Redis, ElasticSearch
- **Cloud & DevOps**: AWS (Lambda, Fargate, EC2, S3, RDS, Amplify, SQS/SNS, EventBridge), Azure (AKS, Key Vault), GCP (Compute Engine, Networking), Cloudflare Workers, Docker, Kubernetes, Helm, Istio, CI/CD, Jenkins, GitHub Actions, SonarCloud, RabbitMQ, Kafka, Message Brokers, Git (GitHub, GitLab)
- **APIs & Payments**: REST, GraphQL, WebSockets, Webhooks, OAuth/JWT, Stripe, Wise
- **Observability & Performance**: Prometheus, Grafana, Jaeger, Elastic APM, PromQL, Distributed Tracing, Load Testing (Locust), Capacity Planning, Queueing Models
- **Architecture & Practices**: System Design, Microservices, API Gateway, Event-Driven Systems, Performance Optimization, Scalability, Cloud Native Architecture, System Architecture Patterns, Hybrid Mobile Architecture, Distributed Systems Architecture
- **Tools**: Jupyter, NetBeans, Eclipse, VS Code, HTC Vive

Projects & Research

Microservices Performance Engineering: Observability, Capacity Modelling & Scaling
Kubernetes, Helm, Istio (Envoy), Prometheus, Grafana, Jaeger, PromQL, Locust, JMT

- Deployed a polyglot gRPC microservices system (Go, Java, .NET, Node.js, Python) on Kubernetes with an Istio service mesh, and provisioned the observability stack (Prometheus, Grafana, Jaeger) via Helm (kube-prometheus-stack).
- Set up zero-code distributed tracing through Envoy sidecars (Zipkin to Jaeger, 100% sampling) and wrote PromQL queries for per-service throughput, mean latency and p95; automated collection of metrics and service dependency graphs into a single script.
- Diagnosed a CrashLoopBackOff caused by CPU throttling that failed gRPC liveness probes on .NET and Node.js services, and retuned container CPU/memory limits to stabilise the cluster.
- Load-tested with Locust from 10 to 200 concurrent users and built a queueing-network capacity model (validated in JMT) that predicted response times within 10% up to 89% bottleneck utilisation and located the frontend's saturation point ($\sim$58 req/s).
- Scaled the bottleneck service horizontally based on the model, cutting mean response time by 79% (199 to 41 ms) and p95 latency by 82% (807 to 144 ms) at 100 concurrent users, with a cost analysis of the added CPU/memory.

Honeypot Analysis and Visualization Tool (FYP)
Data Analysis, MongoDB, Geo-Mapping

- Analyzed security data from various honeypots (Dionaea, Suricata, Kippo).
- Visualized attack data intensity on interactive geographical maps.
- Centralized data ingestion pipeline in MongoDB for increased query performance.

SWAM: Scalable Event-Driven Incident Management System
Quarkus (Jakarta EE, JAX-RS), Angular, PostgreSQL, Kafka, Kubernetes

- Designed a hexagonal (ports-and-adapters) architecture isolating the core domain behind a REST API built with Jakarta EE on Quarkus, using a lightweight CQRS read path.
- Implemented event-driven duplicate detection: the API publishes incident-creation events to a Kafka topic, consumed asynchronously by an independent analyzer service using fuzzy text-similarity matching, reporting back without blocking the write path.
- Enforced role-based access control with incident visibility scoped to departments and transactional write operations.
- Deployed via Docker Compose locally and Kubernetes in production, with the API service horizontally autoscaled.
- Verified the event-driven flow with Testcontainers-based integration tests.

Encrypted Document Vault with Sharing Control
Go (Golang), Angular, RabbitMQ, MongoDB, Docker, WebSockets

- Engineered a secure, distributed document vault platform for sharing sensitive files with full owner control.
- Designed a multi-instance distributed architecture to support horizontal scaling, ensuring system-wide state consistency by using RabbitMQ (AMQP) for reliable event dispatching across instances.
- Implemented a real-time "Eye Tracking System" via WebSockets to instantly notify document owners of current viewers, tracking access by IP address and browser fingerprint.
- Enforced comprehensive security with AES-256 file encryption, configurable burn-once links, and instant system-wide access revocation.

More ML coursework & research: Dry Bean classification with feature selection and PCA (scikit-learn); Python code summarization with a Transformer architecture; static vs dynamic token merging for visual document retrieval.

Achievements & Leadership

Università degli Studi di Firenze Jul 2025 – Sep 2026
Student Representative

- Member of the Commissione Paritetica Docenti-Studenti CdS, serving as a bridge between the student body and faculty.
- Contribute to the evaluation and improvement of teaching quality and academic services within the Software, Science and Technology program.

WomenInTechPK Feb 2023 – Present
Education Mentor

- Lead hands-on sessions and mentoring programs to help participants gain real-world experience with modern tech tools, frameworks, and workflows used in industry and civic tech.

TEN 2017 – 2018
Alumni Relations Manager